

# IT3012 - Lecture 06 - Reasoning and Inference in Intelligent Agents

Jeewaka Perera

# Learning Objectives

By the end of this lecture, you should be able to:

- 1 **Understand** the architecture of a Knowledge-Based Agent.
- 2 **Translate** real-world facts into Propositional and First-Order Logic (FOL).
- 3 **Convert** logical sentences into Conjunctive Normal Form (CNF).
- 4 **Apply** the Resolution rule to mathematically prove logical conclusions.
- 5 **Perform** Unification on FOL variables.
- 6 **Differentiate** between Forward (Data-driven) and Backward (Goal-driven) Chaining.

# Where We Are: From Exploration to Validation

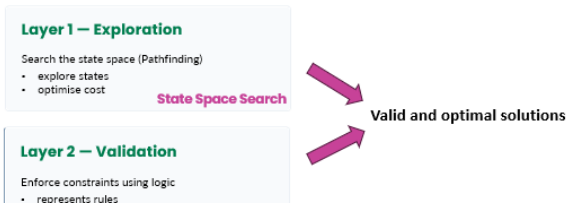
Deterministic Planning in Intelligent Agents has two primary layers:

## Layer 1: Exploration (Weeks 2-3)

- Focuses on searching the state space.
- Goal: Find a path and optimize cost.
- Method: Relaxation (removing constraints to find heuristics).

## Layer 2: Validation (Weeks 4-5)

- Focuses on enforcing constraints using logic.
- Goal: Ensure the chosen path/action is actually allowed.
- Method: Restriction (applying logical safety rules).



# The Wumpus World & Deductive Reasoning

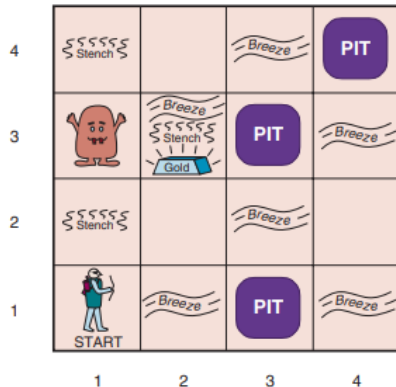
# The Intuition Test

Imagine a simple grid-based game where you are an explorer.

- **The Rule:** If you are standing in a square and feel a *Breeze*, there is a *Pit* in an adjacent square.
- **The Facts:**
  - You are in Square [1,1]. There is NO breeze.
  - You move to Square [2,1]. You FEEL a breeze.

## The Question

Is there a Pit in Square [2,2]? Is there a Pit in [1,2]?



# From Human Intuition to Machine Logic

As a human, you can deduce:

- Because  $[1,1]$  had no breeze,  $[1,2]$  cannot have a pit.
- Therefore, the breeze in  $[2,1]$  must be coming from  $[2,2]$  or  $[3,1]$ .

**The AI Challenge:** How do we teach a computer to do this? We cannot just give it a massive 'if-else' tree for every possible grid combination.

We must give the computer a **Knowledge Base** and an **Inference Engine**.

An intelligent agent needs to know things about the world, and it needs to be able to deduce new things from what it already knows.

## Core Components:

- ① **Knowledge Base (KB):** A set of "sentences" expressed in a formal language. These are the absolute truths the agent knows.
- ② **Inference Engine:** The mathematical algorithms used to derive new sentences from the KB.

# Declarative vs. Procedural Paradigms

## Procedural (Traditional Code)

- You tell the computer *exactly how* to solve the problem step-by-step.
- Focus on control flow ('for' loops, 'if-else').
- Hard to update rules without rewriting logic.

## Declarative (Logic Agents)

- You tell the computer *what* is true, and let it figure out the steps.
- Focus on facts and relationships.
- Easy to update: just add a new fact to the Knowledge Base!

# Reachability vs. Feasibility

Why isn't a search algorithm like A\* enough on its own? Because finding a path does not mean the path is legally allowed to be taken.

## Reachable

- There exists a connected path in the state-space graph from the Start node to the Goal node.

## Feasible

- There exists a solution path that strictly satisfies all logical constraints and rules of the environment.
- *Example:* A path through a maze is *Reachable* for a robot, but if a logical rule states "Do not enter if humans are present," the path may become *Infeasible*.

# The Language of True and False

# What is Propositional Logic?

Propositional Logic (PL) is the simplest form of logic. It deals with **propositions**—statements that are strictly either TRUE or FALSE.

- **Valid Proposition:** "It is raining." (Can be True or False)
- **Valid Proposition:** "The grass is wet." (Can be True or False)
- **Invalid:** "What time is it?" (Not a truth statement)
- **Invalid:** " $x < 5$ " (Depends on  $x$ ; this requires First-Order Logic).

# Syntax: The Symbols of Propositional Logic

We use specific mathematical symbols to connect propositions. Let's look at how an agent evaluates its environment:

| Symbol            | Meaning | Agent Application Example                                                            |
|-------------------|---------|--------------------------------------------------------------------------------------|
| $\neg$            | NOT     | $\neg \text{ManaLow}$ (The agent has sufficient mana)                                |
| $\wedge$          | AND     | $\text{TargetInRange} \wedge \text{SkillReady}$                                      |
| $\vee$            | OR      | $\text{PathBlocked} \vee \text{EnemySpotted}$                                        |
| $\Rightarrow$     | IMPLIES | $\text{EnemySpotted} \Rightarrow \neg \text{MoveForward}$                            |
| $\Leftrightarrow$ | IFF     | $\text{SafeToFarm} \Leftrightarrow \neg \text{EnemySpotted} \wedge \text{HasVision}$ |

# Understanding Implication ( $\Rightarrow$ )

The implication  $P \Rightarrow Q$  (If  $P$ , then  $Q$ ) is often confusing.

- $P$  is the *premise*.  $Q$  is the *conclusion*.
- "If you get 100% on the final ( $P$ ), I will give you an A ( $Q$ )."

**When is  $P \Rightarrow Q$  considered FALSE?**

- Only when  $P$  is True, but  $Q$  is False. (You got 100%, but I didn't give you an A. I lied!)
- If  $P$  is False (you failed the final), the statement  $P \Rightarrow Q$  is still logically True regardless of  $Q$ , because the rule wasn't violated.

# Semantics: Truth Tables

We determine the truth of complex sentences by evaluating every possible combination of their variables.

| $P$ | $Q$ | $\neg P$ | $P \wedge Q$ | $P \vee Q$ | $P \Rightarrow Q$ |
|-----|-----|----------|--------------|------------|-------------------|
| F   | F   | T        | F            | F          | T                 |
| F   | T   | T        | F            | T          | T                 |
| T   | F   | F        | F            | T          | <b>F</b>          |
| T   | T   | F        | T            | T          | T                 |

**Entailment** means that one thing follows absolutely from another.

Notation:  $KB \models \alpha$  (The Knowledge Base entails sentence  $\alpha$ ).

## Definition

$KB \models \alpha$  is true IF AND ONLY IF in every single world where the KB is true,  $\alpha$  is also true.

*Example:*

$KB =$  "It is raining" AND "If it rains, grass is wet."

$\alpha =$  "The grass is wet."

The KB entails  $\alpha$ . There is no universe where the KB is true but the grass is dry.

# Inference vs. Entailment

- **Entailment** ( $KB \models \alpha$ ) is the abstract mathematical truth. It is the destination.
- **Inference** ( $KB \vdash \alpha$ ) is the actual algorithm our computer uses to reach that destination.

An inference algorithm is **Sound** if it only derives true things. (It never lies).

An inference algorithm is **Complete** if it can derive *all* true things. (It never misses a truth).

# Important Logical Equivalences

Just like in algebra, we can rewrite logical sentences into equivalent forms. You must memorize these for AI algorithms:

- **Implication Elimination:**  $P \Rightarrow Q \equiv \neg P \vee Q$
- **Biconditional Elimination:**  $P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$
- **De Morgan's Laws:**
  - $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
  - $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
- **Double Negation:**  $\neg(\neg P) \equiv P$

# Teaching a Computer to Prove Facts

# Basic Inference Rules

Before we look at the ultimate algorithm, let's look at simple human deduction patterns:

## Modus Ponens

- If  $P \Rightarrow Q$  is true, and  $P$  is true, then  $Q$  must be true.

## Modus Tollens

- If  $P \Rightarrow Q$  is true, and  $\neg Q$  is true ( $Q$  is false), then  $\neg P$  must be true.

## And-Elimination

- If  $P \wedge Q$  is true, then  $P$  is true, and  $Q$  is true independently.

# The Problem with Simple Rules

Applying Modus Ponens over and over is a weak search strategy. It can get stuck, and it requires our Knowledge Base to be perfectly structured.

Instead, modern AI uses a single, mathematically complete rule called **Resolution**.

But to use Resolution, we must first translate every sentence in our Knowledge Base into a strict format called **Conjunctive Normal Form (CNF)**.

# Conjunctive Normal Form (CNF)

**CNF** is an "AND of ORs".

Every sentence is a conjunction ( $\wedge$ ) of clauses, where each clause is a disjunction ( $\vee$ ) of literals.

**Valid CNF:**

$$(A \vee B) \wedge (\neg C \vee D \vee E) \wedge (F)$$

**Invalid CNF:**

$$(A \wedge B) \vee C \quad (\text{This is an OR of ANDs})$$

$$A \Rightarrow B \quad (\text{Implications are not allowed})$$

# Converting to CNF (The Steps)

Any logical sentence can be converted to CNF using these 4 steps:

- 1 **Eliminate**  $\Leftrightarrow$ : Replace with  $(A \Rightarrow B) \wedge (B \Rightarrow A)$ .
- 2 **Eliminate**  $\Rightarrow$ : Replace  $A \Rightarrow B$  with  $\neg A \vee B$ .
- 3 **Move**  $\neg$  **inwards**: Use De Morgan's laws so NOTs only apply to single variables (e.g.,  $\neg(A \wedge B) \rightarrow \neg A \vee \neg B$ ).
- 4 **Distribute**  $\vee$  **over**  $\wedge$ : Rewrite  $(A \wedge B) \vee C$  as  $(A \vee C) \wedge (B \vee C)$ .

# The Resolution Rule

Once everything is in CNF, we apply the Resolution Rule.

## The Rule

If a clause contains a literal  $P$ , and another clause contains its exact negation  $\neg P$ , they cancel each other out, and we merge the remaining parts into a new clause.

**Formula:**

$$\frac{(P \vee A) \quad \text{and} \quad (\neg P \vee B)}{(A \vee B)}$$

# Proof by Contradiction

To prove a Goal  $G$  using Resolution, the AI uses **Proof by Contradiction**:

- 1 Take the Goal  $G$ , and negate it ( $\neg G$ ).
- 2 Add  $\neg G$  to the Knowledge Base.
- 3 Convert the entire KB into CNF.
- 4 Apply the Resolution rule repeatedly to smash clauses together.
- 5 If you ever smash a literal  $X$  with  $\neg X$ , they cancel out and leave an **Empty Clause** (a contradiction).
- 6 If you hit a contradiction, the original Goal  $G$  **must be true!**

# Resolution Walkthrough: Step 1 & 2

## The Setup:

- Premise 1: If it rains, the grass is wet ( $R \Rightarrow W$ )
- Premise 2: It is raining ( $R$ )

**Goal:** Prove the grass is wet ( $W$ ).

## Step 1: Negate the Goal and add to KB

Our KB now contains:  $\{R \Rightarrow W, R, \neg W\}$

## Step 2: Convert to CNF

Eliminate the implication in Premise 1:  $R \Rightarrow W \equiv \neg R \vee W$ .

*Our CNF KB:*  $\{(\neg R \vee W), (R), (\neg W)\}$

# Resolution Walkthrough: Step 3

*Current CNF KB:*  $\{(\neg R \vee W), (R), (\neg W)\}$

## Step 3: Resolve!

- Take Clause 1  $(\neg R \vee W)$  and Clause 2  $(R)$ .
- $\neg R$  and  $R$  cancel each other out!
- We generate a new clause:  $(W)$ . Add this to the KB.
- Now take our new clause  $(W)$  and resolve it with our negated goal  $(\neg W)$ .
- $W$  and  $\neg W$  cancel each other out!
- This leaves nothing. An **Empty Clause**.

**Conclusion:** Contradiction found. Therefore, the grass is absolutely wet.

# Adding Variables and Quantifiers

# The Limitations of Propositional Logic

Propositional Logic is great, but it is extremely inefficient for large domains.

If we want to state a general rule like "All humans are mortal," in Propositional Logic we would have to write an infinite number of rules:

- $Human(John) \Rightarrow Mortal(John)$
- $Human(Sarah) \Rightarrow Mortal(Sarah)$
- $Human(David) \Rightarrow Mortal(David) \dots$

We need a language that allows us to use **Variables** representing objects.

# First-Order Logic (FOL)

First-Order Logic looks at the world like a database. It assumes the world contains:

- **Objects:** People, houses, numbers, colors (e.g., *John*, *7*, *Red*).
- **Relations (Predicates):** Properties that are true or false about objects (e.g., *BrotherOf*, *BiggerThan*, *IsRed*).
- **Functions:** Operations that return exactly one object (e.g., *FatherOf(John)* returns a specific person).

- **Constants:** Start with uppercase (*John*, *SriLanka*).
- **Variables:** Start with lowercase ( $x$ ,  $y$ ). They stand in for any object.
- **Atomic Sentences:** A predicate applied to terms.  
E.g.,  $Brother(John, Richard) \rightarrow$  "John is the brother of Richard."  
E.g.,  $Married(Father(John), Mother(John)) \rightarrow$  "John's father is married to John's mother."

# Universal Quantification ( $\forall$ )

The Universal Quantifier  $\forall$  means "For All" or "Every".  
It allows us to apply a rule to the entire universe of objects at once.

**Example:** "All kings are persons."

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$$

**Rule of Thumb:**  $\forall$  is almost always used with the Implication ( $\Rightarrow$ ) operator.  
(If you use  $\forall x \text{ King}(x) \wedge \text{Person}(x)$ , you are claiming that *every single object in the universe* is a king and a person, which is false!)

# Existential Quantification ( $\exists$ )

The Existential Quantifier  $\exists$  means "There exists at least one", "Some", or "Has". It allows us to state that something exists without knowing exactly who or what it is.

**Example:** "King John has a crown on his head."

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

**Rule of Thumb:**  $\exists$  is almost always used with the AND ( $\wedge$ ) operator.

# Translating English to FOL (Examples)

Translating human language into formal logic is a core AI skill.

- **"Brothers are siblings."**

$$\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

- **"One's mother is one's female parent."**

$$\forall m, c \text{ Mother}(m, c) \Leftrightarrow \text{Female}(m) \wedge \text{Parent}(m, c)$$

- **"Not all students take AI."** (There exists some student who does not take AI).

$$\exists x \text{ Student}(x) \wedge \neg \text{Takes}(x, \text{AI})$$

# Nested Quantifiers

We can mix and match quantifiers to create highly complex rules. The order of quantifiers matters completely!

**"Everybody loves somebody."**

For every person  $x$ , there is some person  $y$  that they love.

$$\forall x \exists y \text{ Loves}(x, y)$$

**"There is someone who is loved by everyone."**

There exists a specific person  $y$ , such that every person  $x$  loves them.

$$\exists y \forall x \text{ Loves}(x, y)$$

# Applying Variables to Rules

# How do we infer in FOL?

We have an amazing algorithm for Propositional Logic (Resolution). Can we just use that for FOL?

Yes, through a process called **Propositionalization**. We simply replace every variable with every possible constant in the universe to turn the FOL sentences back into simple propositions.

*The Problem:* If our universe has a lot of objects, substituting every possible constant creates millions of useless sentences. We need a way to only substitute the variables that actually help us prove our goal.

# Substitution and Binding

To use a general rule, we must bind a specific constant to a variable.

**Substitution Notation:**  $\theta = \{x/John, y/Apple\}$

*Reads as: "Substitute John for x, and Apple for y."*

If we apply this substitution  $\theta$  to the sentence  $Eats(x, y)$ , we get:

$Eats(John, Apple)$

**Unification** is the algorithm that takes two logical sentences and finds a substitution  $\theta$  that makes them look exactly identical.

If an AI wants to use the rule:  $\forall x \text{ Knows}(\text{John}, x) \Rightarrow \text{Likes}(\text{John}, x)$

And it has the fact:  $\text{Knows}(\text{John}, \text{Jane})$

It must "Unify" the premise  $\text{Knows}(\text{John}, x)$  with  $\text{Knows}(\text{John}, \text{Jane})$  to find the binding  $\{x/\text{Jane}\}$ .

# Unification: Success vs. Failure

Let's test unifying  $Knows(John, x)$  with different sentences.

| Sentence 1       | Sentence 2            | Result (Substitution $\theta$ ) |
|------------------|-----------------------|---------------------------------|
| $Knows(John, x)$ | $Knows(John, Jane)$   | $\{x/Jane\}$                    |
| $Knows(John, x)$ | $Knows(y, Bill)$      | $\{x/Bill, y/John\}$            |
| $Knows(John, x)$ | $Knows(y, Mother(y))$ | $\{y/John, x/Mother(John)\}$    |
| $Knows(John, x)$ | $Knows(x, Elizabeth)$ | <b>FAILS.</b>                   |

*Why does the last one fail?* Because the variable  $x$  cannot be bound to both 'John' and 'Elizabeth' at the exact same time!

# The Computational Wall: Why Resolution is Slow

Even with Unification, full FOL Resolution is computationally brutal. It is Semi-Decidable (if an answer exists, it will find it, but if no answer exists, the computer might loop forever trying infinite unifications).

To build fast, real-time AI systems (like Expert Systems), we intentionally restrict the types of rules we are allowed to write. We restrict ourselves to **Horn Clauses**.

# Horn Clauses and Definite Clauses

## Definite Clause

A disjunction of literals of which **exactly one** is positive.

Wait, what does that mean? Using Implication Elimination, we know that:

$$(\neg A \vee \neg B \vee C) \equiv (A \wedge B) \Rightarrow C$$

**The Magic:** A Definite/Horn Clause is simply a strict "If-Then" rule!

*Premise<sub>1</sub> ∧ Premise<sub>2</sub> ... ⇒ Conclusion.*

If we build our entire Knowledge Base out of Horn Clauses, inference becomes extremely fast (linear time).

# Fast Inference for Horn Clauses

**Concept:** Start with the facts you know. Look for rules where the premises are satisfied by your facts. Trigger the rule to generate new facts. Repeat until you reach the goal.

- **Direction:** Bottom-Up. (Facts  $\rightarrow$  Rules  $\rightarrow$  Conclusions).
- **Philosophy:** Data-driven. "I have this data, what conclusions can I draw?"
- **Real-world Example:** A doctor examines your symptoms (fever, cough), applies medical rules, and deduces a diagnosis (Flu).

# Forward Chaining in Action

**Rules:** 1.  $A \wedge B \Rightarrow C$

2.  $C \Rightarrow D$

**Known Facts:**  $A, B$

**Execution:**

- ① We know  $A$  and  $B$ .
- ② Rule 1's premise is satisfied. Trigger Rule 1.
- ③ We deduce new fact:  $C$ . Add  $C$  to our KB.
- ④ Rule 2's premise ( $C$ ) is now satisfied. Trigger Rule 2.
- ⑤ We deduce new fact:  $D$ .

# Backward Chaining: Goal-Driven

**Concept:** Start with the Goal you want to prove. Find a rule that has the Goal as its conclusion. Look at that rule's premises and make them your new sub-goals. Repeat until you hit known facts.

- **Direction:** Top-Down. (Goal  $\rightarrow$  Rules  $\rightarrow$  Facts).
- **Philosophy:** Goal-driven. "I want to prove this; what evidence do I need?"
- **Real-world Example:** A detective suspects John committed the crime (Goal). They look for the required evidence (motive, opportunity) to prove it.

# Backward Chaining in Action

**Rules:** 1.  $A \wedge B \Rightarrow C$

2.  $C \Rightarrow D$

**Known Facts:**  $A, B$

**Goal to prove:**  $D$

**Execution:**

- ① I want to prove  $D$ . Rule 2 concludes  $D$ .
- ② To use Rule 2, I need to prove  $C$ . (New sub-goal:  $C$ ).
- ③ Rule 1 concludes  $C$ .
- ④ To use Rule 1, I need to prove  $A$  and  $B$ . (New sub-goals:  $A, B$ ).
- ⑤ I look at my KB.  $A$  and  $B$  are known facts!
- ⑥ The chain is complete.  $D$  is proven.

# Summary: Reasoning and Inference

- **Propositional Logic** uses symbols (AND, OR, NOT) to evaluate facts that are strictly True or False.
- **Resolution** is a complete mathematical algorithm that proves goals by converting everything to CNF and finding contradictions.
- **First-Order Logic (FOL)** introduces Variables and Quantifiers ( $\forall, \exists$ ) to create powerful, universally applicable rules.
- **Unification** binds variables to specific constants so general rules can be applied to specific situations.
- **Horn Clauses** allow us to build fast, linear-time Expert Systems using **Forward** and **Backward Chaining**.

# The Next Frontier: Beyond Binary Constraints

Logical reasoning provides a powerful foundation for Knowledge-Based Agents, allowing them to validate actions and enforce absolute safety conditions.

## The Limitation of Classical Logic

Logical constraints treat every rule as strictly True or strictly False. It is binary.

### But what about the real world?

- What if a path is "slightly slippery"?
- What if an enemy is "moderately close"?
- Next week, we will explore **Fuzzy Logic**, where reasoning is no longer just 0 or 1, but handles gradual truths and uncertainty.